

1. Data Augmentation nima va nima uchun kerak?

Data augmentation bu datani turli usullar bilan sun'iy ko'paytirishdir. biz yaratadigan model uchun data augmentation juda keraklidir. data augmentation modelni turli sharoitlarga, noqulay sharoitlarga moslashishga, ko'nikishga o'rgatadi

2. Mosaic augmentation nima uchun detection uchun ayniqsa foydali?
Mosaic augmentation bu 4ta rasmni bitta rasimga birlashtirish.

3. Roboflow Auto-Label va Manual Annotation farqi nima?

4. Dataset Version nima? Nima uchun versiya yaratamiz?

5. Roboflow API va ZIP usulining farqi va afzalliklari?

6. YOLOv8 training da augment=True nima beradi?

1. Data Augmentation nima va nima uchun kerak?

Data Augmentation — mavjud rasmlarga turli o'zgartirishlar qo'llab, train datasetning sun'iy ravishda xilma-xilligini oshirish usuli.

Data — ma'lumot.

Augmentation — kengaytirish, ko'paytirish yoki yangi ko'rinishlar hosil qilish.

Masalan, train datasetda 500 ta rasm mavjud bo'lsa, har bir rasmning:

- chap-o'ng akslantirilgan;
- yorqinligi o'zgartirilgan;
- kattalashtirilgan;
- kichraytirilgan;
- biroz burilgan;
- bir qismi kesilgan

ko'rinishlari training vaqtida hosil qilinishi mumkin.

Bu jarayonda 500 ta original rasm diskda albatta 5000 ta alohida faylga aylantirilishi shart emas. YOLO training vaqtida rasmlarni tasodifiy o'zgartirib, har epochda bir rasmni turlicha ko'rinishda modelga berishi mumkin. Ultralytics hujjatlariga ko'ra, augmentation datasetni yangi rasm yig'masdan kengaytirish, overfittingni kamaytirish va modelning real sharoitlarga generalizatsiyasini yaxshilash uchun ishlatiladi.

Oddiy misol

Original rasmda avtomobil quyidagi holatda bo'lsin:

Kunduzgi yorug'lik

Avtomobil rasmning o'ng tomonida

Avtomobil katta ko'rinishda

Kamera to'g'ri qaragan

Augmentationdan keyin shu rasm:

yorqinligi pasaytirilgan;

chap-o'ng akslantirilgan;

avtomobil kichiklashtirilgan;

rasm biroz burilgan;

rang to'yinganligi o'zgartirilgan

ko'rinishda trainingga berilishi mumkin.

Model faqat bitta avtomobil rasmini yodlab olmaydi. Avtomobil turli joylashuv, o'lcham va yoritishda ham avtomobil ekanini o'rganadi.

Asosiy maqsadlari

Datasetni xilma-xil qilish

Datasetda rasm soni kam bo'lsa, model bir xil fon, yoritish va kamera holatiga haddan tashqari moslashib qolishi mumkin.

Masalan, barcha yuk mashinalari faqat kunduzgi yo'lda olingan bo'lsa, model tungi yuk mashinasini aniqlashda qiynaladi. Brightness, HSV va boshqa rang o'zgartirishlari modelga turli yoritish sharoitlarini ko'rsatadi.

Overfittingni kamaytirish

Overfitting — model train rasmlarni juda yaxshi yodlab, yangi rasmlarda yomon ishlaydigan holat.

Augmentation har epochda rasmni biroz boshqacha ko'rsatgani sababli, model aniq piksel joylashuvini yodlashdan ko'ra obyektning umumiy xususiyatlarini o'rganishga majbur bo'ladi. Ultralytics rasmiy qo'llanmasida augmentation modelning train rasmlaridagi aniq belgilarni yodlab qolish ehtimolini kamaytirishi ta'kidlangan.

Generalizationni yaxshilash

Generalization — modelning trainingda ko'rmagan yangi rasmlarda ham to'g'ri ishlash qobiliyati.

Real sharoitda obyekt:

- turli o'lchamda;
- turli masofada;
- turli burchakda;

- qorong‘i yoki yorug‘ muhitda;
- qisman yopilgan;
- rasmning turli joylarida

uchrashi mumkin.

Augmentation shu holatlarning ayrimlarini sun‘iy yaratadi.

Kam datasetdan samaraliroq foydalanish

Yangi rasm yig‘ish va annotation qilish ko‘p vaqt talab qiladi. Augmentation mavjud ma‘lumotdan ko‘proq variant hosil qilishga yordam beradi.

Lekin 100 ta rasmga augmentation qo‘llash 10 000 ta haqiqiy va xilma-xil rasmning to‘liq o‘rnini bosmaydi. Augmentation mavjud ma‘lumotni boyitadi, lekin datasetda umuman mavjud bo‘lmagan yangi obyekt yoki yangi real holatni to‘liq yaratib bera olmaydi.

Object detectionda rasm bilan birga bounding box ham o‘zgaradi

Classification vazifasida faqat rasm o‘zgartirilishi mumkin. Object detectionda esa rasm bilan birga bounding box koordinatalarini ham o‘zgartirish shart.

Masalan, avtomobil rasmning chap tomonida bo‘lsa va rasm gorizontal akslantirilsa, avtomobil o‘ng tomonga o‘tadi. Bounding box eski chap tomonda qolib ketsa, label noto‘g‘ri bo‘lib qoladi.

To‘g‘ri pipeline:

Rasm aylantiriladi

+

Bounding box ham shu aylanishga mos o‘zgartiriladi

Ultralytics augmentation pipeline'i rotation, translation, scaling, shearing va perspective o'zgarishlarini rasm bilan birga bounding box, segment va keypoint ma'lumotlariga ham qo'llaydi.

YOLO'da asosiy augmentation turlari

Parametr	O'zbekcha mazmuni	Vazifasi
fliplr	Chap-o'ng akslantirish	Obyektni ikki yo'nalishda tanishga yordam beradi
flipud	Yuqori-past akslantirish	Obyekt teskari holatda ham uchrasa foydali
degrees	Burilish darajasi	Kamera yoki obyekt burchagi o'zgarishini yaratadi
translate	Siljitish	Obyektni rasmning boshqa joyiga ko'chiradi
scale	Masshtabni o'zgartirish	Obyektni yaqin yoki uzoq ko'rinishda beradi
shear	Qiyshaytirish	Geometrik deformatsiya yaratadi
perspective	Perspektivani o'zgartirish	Kamera qarash burchagini taqlid qiladi
hsv_h	Rang tusini o'zgartirish	Turli rang sharoitlarini yaratadi
hsv_s	Rang to'yinganligini o'zgartirish	Rang kuchini o'zgartiradi
hsv_v	Yorqinlikni o'zgartirish	Yorug' va qorong'i holatlarni yaratadi
mosaic	To'rtta rasmni birlashtirish	Turli masshtab va kontekst yaratadi
mixup	Ikki rasmni aralashtirish	Overfittingni kamaytirishga yordam beradi

Ultralytics training transformlari Mosaic, MixUp, Random Perspective, rang o'zgartirishlari va boshqa usullardan tashkil topishi mumkin.

Har qanday augmentation foydali emas

Augmentation real vazifaga mantiqan mos bo'lishi kerak.

Masalan, yo'ldagi avtomobilni yuqori-past aylantirish:

`flipud=1.0`

real hayotga mos kelmaydi. Chunki avtomobil odatda osmonda teskari yurmaydi.

Lekin mikroskop ostidagi hujayra yoki konveyerdagi detal istalgan yoʻnalishda turishi mumkin. Bunday vazifada vertikal flip foydali boʻlishi mumkin.

Yozuv yoki raqam aniqlashda gorizontal flip xavfli boʻlishi mumkin. Akslantirilgan matn real matn emas va labelning maʼnosi buzilishi mumkin.

Juda kuchli augmentation zarari

Haddan tashqari kuchli augmentation quyidagi muammolarni keltirib chiqarishi mumkin:

- obyekt shakli real holatdan juda uzoqlashadi;
- bounding boxning katta qismi kesilib qoladi;
- classni ajratuvchi muhim rang yoʻqoladi;
- model haqiqiy rasmdan koʻra sunʼiy tasvirlarga moslashadi;
- train loss sekin kamayadi;
- validation natijasi yomonlashadi.

Shuning uchun augmentation miqdori validation natijasi bilan tekshirib boriladi.

2. Mosaic augmentation detection uchun nima sababdan ayniqsa foydali?

Mosaic augmentation — toʻrtta turli training rasmini bitta yangi rasm ichiga birlashtirish usuli.

Mosaic — bir nechta boʻlakdan tashkil topgan umumiy tasvir.

Oddiy koʻrinish:

1-rasm car	2-rasm person
3-rasm dog	4-rasm bus

Toʻrtta rasmning bounding boxlari ham yangi birlashtirilgan koordinatalarga mos ravishda qayta hisoblanadi.

Ultralytics'ning joriy implementationida Mosaic tasodifiy tanlangan toʻrtta rasmni birlashtiradi. `mosaic=1.0` har bir training namunasi uchun Mosaic qoʻllash ehtimolini 100 foizga, `mosaic=0.0` esa uni oʻchirishga mos keladi.

Kichik obyektlarni oʻrganishga yordam beradi

Toʻrtta katta rasm bitta training rasmiga joylashtirilganda, har bir rasmning obyektlari nisbatan kichik koʻrinishi mumkin.

Masalan, original rasmda avtomobil katta boʻlsa, Mosaic ichida u kichikroq joyni egallaydi. Model shu orqali bir xil classni:

- katta;
- oʻrta;
- kichik

masshtabda koʻrishga odatlanadi.

Bu uzoqdagi piyoda, kichik transport vositasi, qush, yoʻl belgisi kabi obyektlar uchun foydali. Ultralytics rasmiy qoʻllanmasi Mosaic'ni small-object detection, ya'ni kichik obyektlarni aniqlash va kontekstni oʻrganish uchun ayniqsa samarali deb tavsiflaydi.

Bitta training namunasida ko‘proq obyekt paydo bo‘ladi

Oddiy rasmda bitta yoki ikkita obyekt bo‘lishi mumkin.

Mosaic natijasida:

1-rasm: 2 ta car

2-rasm: 4 ta person

3-rasm: 1 ta bus

4-rasm: 3 ta dog

birlashtirilsa, yangi rasmda jami 10 ta obyekt qatnashadi.

Model bitta forward pass davomida:

- ko‘proq bounding box;
- ko‘proq class;
- ko‘proq masshtab;
- ko‘proq fon

bilan ishlaydi.

Bu kichik datasetdan samaraliroq foydalanishga yordam beradi.

Turli kontekstlarni yaratadi

Context — obyekt atrofidagi muhit va fon.

Original datasetda avtomobil doim yo‘lda turgan bo‘lishi mumkin.

Mosaic ichida uning yonida boshqa rasmning binosi, odam yoki boshqa fon paydo bo‘ladi.

Model obyektни faqat tanish fon bilan bog‘lash o‘rniga, obyektning o‘z shakliga ko‘proq e’tibor berishi mumkin.

Masalan:

car faqat asfalt yoʻlda boʻladi

kabi notoʻgʻri bogʻlanishni kamaytirishga yordam beradi.

Qisman yopilish holatini yaratadi

Mosaic rasm chegaralari va tasodifiy markaz joylashuvi sababli ayrim obyektlar:

- toʻliq koʻrinishi;
- rasm chetida qolishi;
- qisman kesilishi;
- boshqa obyektlarga yaqin turishi

mumkin.

Bu **occlusion**, yaʼni obyektning qisman yopilishi yoki toʻliq koʻrinmasligi holatini oʻrganishga yordam beradi.

Real koʻchada avtomobilning bir qismi boshqa avtomobil bilan, insonning bir qismi daraxt yoki ustun bilan yopilishi mumkin.

Har epochda yangi kombinatsiyalar hosil boʻladi

Datasetda 100 ta rasm boʻlsa, Mosaic har safar toʻrtta rasmning boshqa kombinatsiyasini tanlashi mumkin.

Shu sababli bir xil datasetdan koʻplab turli training kompozitsiyalari hosil boʻladi. Bu modelning aynan original rasmlarni yodlashini qiyinlashtiradi.

Mosaicʻning kamchiliklari

Mosaic har doim ham foydali emas.

Sun'iy kompozitsiya hosil qiladi

To'rtta bir-biriga aloqasiz rasm bitta tasvirga birlashtiriladi. Bunday keskin chegaralar real hayotda uchramasligi mumkin.

Trainingni murakkablashtiradi

Model bir rasmda juda ko'p obyekt, fon va masshtabni ko'radi. Bu trainingning dastlabki bosqichlarida foydali, lekin yakuniy bosqichda modelning oddiy real rasmlarga moslashishini qiyinlashtirishi mumkin.

Ultralytics shu sababli `close_mosaic` parametrini beradi. U training tugashidan oldingi belgilangan epochlar davomida Mosaic'ni o'chirib, modelni oddiy rasmlar bilan barqarorlashtiradi.

Masalan, 50 epoch trainingda:

```
epochs=50
```

```
close_mosaic=10
```

yozilsa, oxirgi 10 epochda Mosaic o'chiriladi.

5 epochlik juda qisqa sinovda Mosaic'ni oxirgi epochda o'chirish uchun:

```
epochs=5
```

```
mosaic=1.0
```

```
close_mosaic=1
```

ishlatilishi mumkin.

Mosaic training oxirigacha saqlanishi kerak bo'lsa:

```
close_mosaic=0
```

qo'llanadi. Rasmiy hujjatda `close_mosaic=0` Mosaic'ni yakunda avtomatik o'chirish funksiyasini bekor qilishi ko'rsatilgan.

Juda kichik datasetda bir rasm takrorlanishi mumkin

Ultralytics hujjatiga ko'ra, dataset juda kichik bo'lsa, ayni rasm bitta Mosaic ichida bir necha marta tanlanishi mumkin.

Bu xilma-xillikni kamaytiradi. Shu sababli Mosaic yangi real rasm yig'ish o'rnini to'liq bosa olmaydi.

3. Roboflow Auto-Label va Manual Annotation farqi

Manual Annotation

Manual Annotation — rasm yoki videodagi obyektlarni inson tomonidan qo'lda belgilash.

Object detectionda annotator:

1. rasmni ochadi;
2. obyekt atrofida bounding box chizadi;
3. class nomini tanlaydi;
4. annotationni tekshiradi;
5. saqlaydi.

Roboflow Annotate bounding box va polygonlarni qo'lda chizish imkonini beradi. Roboflow rasmiy hujjatlarida manual drawing va AI Labeling alohida annotation usullari sifatida ko'rsatilgan.

Afzalligi

Inson loyiha classlarini yaxshi tushunsa, juda aniq annotation yaratilishi mumkin.

Masalan:

small_crack
deep_crack
surface_scratch
broken_connector

kabi bir-biriga o'xshash, lekin ma'nosi boshqa classlarni mutaxassis yaxshiroq ajratadi.

Kamchiligi

Har bir obyektga alohida box chizish ko'p vaqt talab qiladi.

1000 ta rasmda o'rtacha 10 tadan obyekt bo'lsa:

$1000 \times 10 = 10\ 000$ ta bounding box

chizish talab qilinadi.

Shuningdek, annotatorlar orasida farq bo'lishi mumkin:

- biri boxni juda keng chizadi;
- biri juda tor chizadi;
- biri kichik obyektни belgilaydi;
- boshqasi belgilamaydi.

Shuning uchun annotation guideline, ya'ni aniq belgilash yo'riqnomasi tuzilishi kerak.

Roboflow Auto-Label

Auto-Label — AI modeli yordamida rasmlarga avtomatik annotation yaratish.

Roboflow Auto Label common object, ya'ni umumiy ko'rinishi tushunarli obyektlarni matnli class tavsifi orqali topishga yordam

beradi. Rasmiy hujjatlarda transport vositalari, odamlar, umumiy yoriqlar va umumiy mahsulotlar Auto Label uchun mos misollar sifatida keltirilgan.

Jarayon taxminan quyidagicha bo‘ladi:

Rasmlar yuklanadi

- class va uning tasviri yoziladi
- Auto Label kichik namunada tekshiriladi
- model obyektlarni topadi
- bounding box yoki mask hosil qiladi
- inson natijani tekshiradi

Masalan, class:

forklift

tavsifi:

industrial forklift vehicle

ko‘rinishida berilishi mumkin.

Foundation model — juda katta va xilma-xil ma’lumotlarda o‘qitilgan, turli vazifalarga moslasha oladigan umumiy model.

Roboflow Auto Label object detection uchun Grounding DINO, segmentation uchun Grounded SAM kabi foundation modellardan foydalanishi mumkin.

Auto-Label afzalliklari

Tezlik

Minglab rasmlarga dastlabki annotation qisqa vaqt ichida yaratilishi mumkin.

Boshlang'ich dataset yaratish

Common classlar uchun noldan box chizish o'rniga model taklifi olinadi va keyin tuzatiladi.

Inson mehnatini kamaytirish

Annotatorning vazifasi:

boxni noldan chizish

o'rniga:

tayyor boxni tekshirish va tuzatish

ga aylanadi.

Auto-Label cheklovlari

Auto-Label annotationi avtomatik ravishda 100 foiz to'g'ri deb qabul qilinmaydi.

Quyidagi xatolar uchrashi mumkin:

- obyektни topmay qolish;
- noto'g'ri class berish;
- boxni noto'g'ri chizish;
- kichik obyektни o'tkazib yuborish;
- fonni obyekt deb belgilash;
- juda o'xshash classlarni ajrata olmaslik.

Roboflow hujjatida Auto Label umumiy obyektlarda yaxshi ishlashi, lekin o'xshash obyektlarning juda maxsus variantlarini farqlashda cheklanishi ko'rsatilgan. Masalan, oddiy yoriqni topishi mumkin, ammo turli xil maxsus yoriq turlarini ishonchli ajratmasligi mumkin.

Label Assist bilan farqi

Roboflow'da **Label Assist** degan alohida model-assisted labeling imkoniyati ham mavjud.

Label Assist:

- oldin train qilingan o'z modelidan;
- Roboflow Universe'dagi public modeldan

foydalanib, rasm ochilganda prediction yaratadi. Topilgan obyektlar annotation sifatida qo'shiladi.

Farqi:

Auto Label:

umumiy foundation model va matnli tavsif yordamida annotation

Label Assist:

oldindan train qilingan maxsus yoki public model yordamida annotation

Masalan, loyihada automobile va truck uchun birinchi 300 ta rasm qo'lda belgilanadi. YOLO modeli train qilinadi. Keyingi 2000 ta rasm uchun shu train qilingan model Label Assist sifatida ishlatiladi.

Bu maxsus classlarda umumiy foundation modeldan aniqroq bo'lishi mumkin.

Eng to'g'ri amaliy yondashuv

Haqiqiy loyihada quyidagi hybrid jarayon samarali bo'ladi:

1. Bir qism rasm manual annotation qilinadi
2. Annotationlar tekshiriladi

3. Kichik model train qilinadi
4. Model qolgan rasmlarga auto-label beradi
5. Inson barcha natijalarni tekshiradi
6. Xatolar tuzatiladi
7. To'liq datasetda model retrain qilinadi

Hybrid — qo'lda va avtomatik usulni birlashtirgan yondashuv.

Qisqa taqqoslash

Xususiyat	Manual Annotation	Roboflow Auto-Label
Annotationni kim yaratadi?	Inson	AI model
Tezlik	Sekinroq	Tezroq
Maxsus classlar	Yaxshi	Qiyin bo'lishi mumkin
Katta dataset	Juda ko'p vaqt oladi	Qulayroq
Xato ehtimoli	Inson xatosi	Model xatosi
Tekshiruv	Zarur	Albatta zarur
Boshlang'ich mehnat	Yuqori	Kamroq
Eng yaxshi qo'llanish	Murakkab va maxsus obyektlar	Umumiy va aniq ko'rinadigan obyektlar

4. Dataset Version nima va nima uchun yaratiladi?

Dataset Version — datasetning ma'lum bir vaqtdagi o'zgarmaydigan nusxasi.

Version — versiya, ya'ni ma'lum holat yoki nashr.

Roboflow'da Dataset Version quyidagilarni o'z ichiga oladi:

- rasmlar;
- annotationlar;
- train, validation va test bo'linishi;
- preprocessing sozlamalari;

- augmentation sozlamalari.

Roboflow rasmiy hujjatiga ko'ra, version datasetning ma'lum vaqtdagi snapshoti, ya'ni muzlatilgan holati hisoblanadi. Version yaratilganda preprocessing va augmentation ham belgilanishi mumkin.

Project va Version farqi

Project — doimiy ravishda o'zgarib turadigan ish maydoni.

Project ichida:

- yangi rasm qo'shiladi;
- noto'g'ri annotation tuzatiladi;
- class o'zgartiriladi;
- rasm o'chiriladi;
- yangi annotation tasdiqlanadi.

Version esa yaratilgan vaqtdagi holatni saqlab qoladi.

Masalan:

Version 1

500 ta rasm
2 ta class
80/10/10 split
resize: 640×640
augmentation: flip

Version 2

700 ta rasm
2 ta class
noto'g'ri 50 ta annotation tuzatilgan

80/10/10 split
resize: 640×640
augmentation: flip + brightness

Version 3

900 ta rasm
3 ta class
yangi bus classi qo'shilgan
70/20/10 split
resize: 640×640
augmentation: mosaic yo'q

Har bir version alohida tajriba sifatida ishlatiladi.

Version nima uchun muhim?

Reproducibility

Reproducibility — ayni tajribani keyinchalik bir xil sharoitda qayta bajarish imkoniyati.

Model yaxshi natija berganda qaysi dataset ishlatilgani ma'lum bo'lishi kerak.

Faqat:

datasetim bilan train qildim

deyish yetarli emas. Chunki dataset keyinchalik o'zgargan bo'lishi mumkin.

To'g'ri yozuv:

Roboflow project: vehicle-detection

Dataset version: 4

Format: YOLOv8

Model: yolov8m.pt

Epochs: 50

Image size: 640

bo'ladi.

Roboflow'da yaratilgan version keyingi annotation tahrirlaridan mustaqil saqlanadi. Bu model taqqoslash va tajribani qayta tiklashga yordam beradi.

Modellarni adolatli taqqoslash

Masalan, YOLOv8n va YOLOv8m taqqoslanayotgan bo'lsa, ikkalasi ham aynan bir xil dataset versionida train qilinishi kerak.

Aks holda:

YOLOv8n – Version 1, 500 rasm

YOLOv8m – Version 3, 900 rasm

bo'lsa, natijadagi farq faqat modeldan kelib chiqqanini aniqlab bo'lmaydi.

Annotation o'zgarishlarini nazorat qilish

Version 1'da annotation xatolari bo'lishi mumkin.

Version 2'da:

- noto'g'ri boxlar tuzatiladi;
- yetishmagan obyektlar belgilanadi;
- class IDlar to'g'rilanadi.

Keyin Version 1 va Version 2 model natijalari taqqoslanadi.

Annotation sifati modelga qancha ta'sir qilgani ko'rinadi.

Preprocessing va augmentationni nazorat qilish

Preprocessing — trainingdan oldin rasmlarga doimiy tayyorlov amallarini qo‘llash.

Masalan:

- `resize`;
- `auto-orient`;
- `class remapping`;
- `grayscale`.

`Resize` — rasm o‘lchamini o‘zgartirish.

`Auto-orient` — rasm orientatsiyasini metadata asosida to‘g‘rilash.

Version orqali:

`Version 1: 640×640 resize`

`Version 2: 416×416 resize`

natijalarini taqqoslash mumkin.

Roboflow Version yaratish jarayonida `train/test/valid split`, `preprocessing` va `augmentations` belgilanishi mumkin.

Production modelni qayta tiklash

Production — modelning real ishlaydigan tizimga joylashtirilgan holati.

Bir yil o‘tgach production modelni qayta train qilish talab qilinsa, qaysi dataset versionidan foydalanilgani ma’lum bo‘lishi kerak.

Shuning uchun model bilan birga quyidagilar saqlanadi:

`model checkpoint`;

`dataset version`;

```
training config;  
library version;  
metrics;  
random seed.
```

Version o‘zgarmaydi

Version yaratilgandan keyin projectga yangi rasm qo‘shilishi eski versionni o‘zgartirmaydi. Yangi o‘zgarishlar bilan yangi version yaratiladi. Roboflow buni reproducibility va model comparison uchun asosiy sabab sifatida ko‘rsatadi.

5. Roboflow API va ZIP usulining farqi va afzalliklari

Roboflow datasetini ikki asosiy usul bilan olish mumkin:

1. ZIP faylni brauzer orqali yuklab olish
2. API yoki berilgan kod orqali avtomatik yuklash

Roboflow interfeysi datasetni ZIP sifatida bevosita yuklash yoki Python kodi, `curl` buyrug‘i va direct download link olish imkonini beradi.

ZIP usuli

ZIP usulida:

1. Roboflow Version ochiladi.
2. Kerakli format tanlanadi.
3. Download ZIP bosiladi.
4. Fayl kompyuterga yuklanadi.
5. ZIP qo‘lda ochiladi.

6. Dataset pathi kodda ko'rsatiladi.

Masalan:

vehicle-detection-4.zip

ochilgandan keyin:

vehicle-detection-4/

```
|— train/  
|— valid/  
|— test/  
|— data.yaml  
|— README files
```

hosil bo'ladi.

ZIP afzalliklari

- ishlatish juda sodda;
- Python kodi talab qilinmaydi;
- API keyni notebookga kiritish shart emas;
- yuklangandan keyin internetsiz ishlash mumkin;
- dataset fayllarini ko'z bilan tekshirish oson;
- bir martalik kichik loyiha uchun qulay.

ZIP kamchiliklari

- har safar qo'lda yuklash kerak;
- Colabga alohida upload qilish kerak;
- qaysi version yuklanganini adashtirish mumkin;
- eski ZIP bilan ishlab qolish ehtimoli bor;
- avtomatlashtirilgan pipeline uchun noqulay;
- boshqa kompyuterda tajribani qayta bajarishda faylni yana topish kerak.

Roboflow eksport ZIP faylini ayni version va format uchun qayta ishlatishi mumkin; yuklangan rasm soni mos kelmasa, yangi version yaratib qayta eksport qilish tavsiya qilinadi. Yuklab olinadigan rasmlar training tezligi va yetarli sifat oʻrtasidagi muvozanat uchun siqilishi ham mumkin.

API usuli

API — Application Programming Interface — bir dasturga boshqa dastur bilan kod orqali muloqot qilish imkonini beradigan interfeys.

Roboflow API yoki Python SDK yordamida:

- workspace tanlanadi;
- project tanlanadi;
- version koʻrsatiladi;
- export format tanlanadi;
- dataset avtomatik yuklanadi.

SDK – **Software Development Kit** — maʼlum xizmat bilan ishlash uchun tayyor kutubxona va vositalar toʻplami.

Oddiy Python koʻrinishi:

```
from roboflow import Roboflow

rf = Roboflow(api_key="ROBOFLOW_API_KEY")

project = (
    rf.workspace("workspace-name")
        .project("project-name")
)

dataset = (
```

```
project.version(4)
    .download("yolov8")
)
```

Bu kod misol sifatida beriladi. Workspace, project, version va format nomlari Roboflow bergan export kodidan olinishi kerak.

Roboflow REST API tuzilishi workspace → project → version → format ketma-ketligiga ega. Muayyan version va format endpointi zipped dataset uchun download link qaytaradi.

API afzalliklari

Avtomatlashtirish

Notebook boshidan ishga tushirilganda dataset avtomatik yuklanadi.

Notebookni ochish

→ kodni run qilish

→ Version 4 avtomatik yuklanadi

→ training boshlanadi

Qoʻlda ZIP topish va ochish talab qilinmaydi.

Reproducibility

Kodda aniq version yoziladi:

```
project.version(4)
```

Shu sababli boshqa kompyuter yoki Colab ham ayni Version 4'ni yuklaydi.

Google Colab uchun qulaylik

Colab sessiyasi oʻchirilganda lokal fayllar yoʻqoladi. API kodi qayta run qilinib dataset yana yuklanadi.

Pipeline va CI/CD uchun mos

CI/CD – Continuous Integration / Continuous Delivery — kod, test va deployment jarayonlarini avtomatlashtirish tizimi.

Training serverida datasetni API orqali olish mumkin. Har safar qoʻlda ZIP yuklash talab qilinmaydi.

Formatni kodda aniq belgilash

Masalan:

yolov8
coco
voc

formatlaridan keraklisi avtomatik tanlanadi.

Roboflow 50 dan ortiq annotation formatlariga eksport qilish va kod snippet orqali yuklash imkonini beradi.

API kamchiliklari

- internet kerak;
- API key talab qilinadi;
- notoʻgʻri key yoki ruxsat xatosi chiqishi mumkin;
- maxfiy keyni GitHubʻga yuklab yuborish xavfi bor;
- paket versiyasi yoki API oʻzgarishi kodga taʻsir qilishi mumkin;
- boshlangʻich foydalanuvchi uchun ZIPʻdan murakkabroq.

API key xavfsizligi

API key kod ichida ochiq yozilmasligi maʼqul:

```
api_key="rf_123456789..."
```

Ayniqsa notebook GitHub'ga yuklansa, begona odam keydan foydalanishi mumkin.

Environment variable ishlatiladi:

```
import os
from roboflow import Roboflow

ROBOFLOW_API_KEY = os.getenv(
    "ROBOFLOW_API_KEY"
)

if not ROBOFLOW_API_KEY:
    raise ValueError(
        "ROBOFLOW_API_KEY topilmadi."
    )

rf = Roboflow(
    api_key=ROBOFLOW_API_KEY
)
```

Environment variable — maxfiy yoki tizimga tegishli qiymatni koddan tashqarida saqlash usuli.

Google Colab'da keyni Secrets bo'limida saqlash mumkin.

API va ZIP orasidagi eng muhim farq

Ayni dataset versioni va ayni export format tanlansa, dataset mazmuni asosan bir xil bo'ladi. Farq ko'proq uni qanday olishda:

ZIP:

inson brauzer orqali qo'lda yuklaydi

API:

kod version va formatni ko'rsatib avtomatik yuklaydi

Qaysi holatda qaysi biri?

Holat	Ma'qul usul
Bir marta lokal tajriba	ZIP
Yangi boshlang'ich mashq	ZIP
Google Colab notebook	API
Ko'p marta training	API
Jamoaviy loyiha	API
Avtomatik training serveri	API
Internet keyinchalik bo'lmaydi	ZIP
API keyni ishlatish mumkin emas	ZIP
Bir xil tajribani qayta bajarish	API

6. YOLOv8 trainingda augment=True nima beradi?

Bu qismda muhim farq mavjud.

Joriy Ultralytics trainingda augmentation qanday boshqariladi?

Ultralytics'ning joriy rasmiy training sozlamalarida `augment=True` training uchun umumiy asosiy kalit sifatida ko'rsatilmagan. Training augmentationlari alohida parametrlar bilan boshqariladi:

```
mosaic=1.0  
fliplr=0.5  
flipud=0.0  
degrees=0.0  
translate=0.1  
scale=0.5  
shear=0.0
```

```
perspective=0.0  
mixup=0.0  
hsv_h=0.015  
hsv_s=0.7  
hsv_v=0.4
```

Ultralytics training pipeline'i ushbu hyperparameterlar asosida Mosaic, MixUp, Random Perspective, flip va rang transformlarini qo'llaydi.

Shu sababli training kodida:

```
model.train(  
    data="data.yaml",  
    epochs=50,  
    augment=True,  
)
```

yozishdan ko'ra, kerakli augmentationlarni aniq parametrlar orqali boshqarish to'g'riroq bo'ladi.

augment=True qayerda rasmiy ko'rsatilgan?

Joriy Ultralytics konfiguratsiyasida augment=True:

- prediction;
- validation

uchun **Test-Time Augmentation** — TTA ni yoqadi.

Test-Time Augmentation — prediction vaqtida bitta rasmning bir nechta o'zgartirilgan variantlarini modeldan o'tkazib, natijalarni birlashtirish usuli.

Rasmiy konfiguratsiyada `augment=False` predict va validation uchun default bo'lib, `True` qilinganda detection barqarorligi yoki aniqligi biroz yaxshilanishi mumkin, lekin inference sekinlashadi.

Prediction misoli:

```
results = model.predict(  
    source="image.jpg",  
    augment=True,  
)
```

Validation misoli:

```
metrics = model.val(  
    data="data.yaml",  
    augment=True,  
)
```

Bu training augmentationidan boshqa jarayon.

Training augmentation va TTA farqi

Xususiyat	Training augmentation	Test-Time Augmentation
Qachon ishlaydi?	Model train qilinayotganda	Validation yoki predictionda
Weightlar o'zgaradimi?	Ha	Yo'q
Maqsad	Generalizationni yaxshilash	Predictionni mustahkamlash
Tezlik	Trainingni og'irlashtirishi mumkin	Inference'ni sekinlashtiradi
Boshqarish	mosaic, fliplr, scale va boshqalar	augment=True
Natija	Yaxshiroq o'rgangan model	Bir rasm uchun ko'proq hisoblash

`amp=True` bilan adashtirmaslik kerak

Training kodida ko'p uchraydigan:

`amp=True`

`augmentation` emas.

AMP — Automatic Mixed Precision — GPU trainingda FP16 va FP32 hisoblash formatlarini birgalikda ishlatish.

Uning vazifasi:

- GPU xotirasini kamaytirish;
- trainingni tezlashtirish;
- zamonaviy GPU'dan samaraliroq foydalanish.

Ultralytics rasmiy konfiguratsiyasida `amp=True` training uchun default bo'lib, `mixed-precision` trainingni yoqadi.

Farqi:

`augmentation:`
`rasmni o'zgartiradi`

AMP:
sonli hisoblash aniqligi va tezligini boshqaradi

YOLOv8 detection uchun tavsiya qilinadigan kod

Avtomobil va yuk mashinasi kabi odatiy object detection datasetida konservativ, ya'ni haddan tashqari kuchli bo'lmagan sozlama:

```
from ultralytics import YOLO
```

```
model = YOLO("yolov8m.pt")
```

```
results = model.train(  
    data="data.yaml",  
    epochs=50,  
    imgsz=640,
```

```
batch=8,  
  
# Geometrik augmentation  
degrees=5.0,  
translate=0.10,  
scale=0.50,  
shear=0.0,  
perspective=0.0,  
  
# Akslantirish  
fliplr=0.5,  
flipud=0.0,  
  
# Rang va yorqinlik  
hsv_h=0.015,  
hsv_s=0.7,  
hsv_v=0.4,  
  
# Birlashtiruvchi augmentationlar  
mosaic=1.0,  
mixup=0.0,  
close_mosaic=10,  
  
# Augmentation emas  
amp=True,  
)
```

Bu qiymatlar yakuniy universal sozlama emas. Datasetga qarab validation natijasi orqali tanlanadi.

5 epochlik qisqa training uchun

5 epoch juda qisqa bo'lgani sababli oxirgi 10 epochda Mosaic'ni o'chirish mantiqan mos kelmaydi. Qisqa sinov uchun:

```
results = model.train(  
    data="data.yaml",  
    epochs=5,  
    imgsz=640,  
    batch=2,  
  
    mosaic=1.0,  
    close_mosaic=1,  
  
    flipplr=0.5,  
    flipud=0.0,  
  
    degrees=5.0,  
    translate=0.1,  
    scale=0.5,  
  
    hsv_h=0.015,  
    hsv_s=0.7,  
    hsv_v=0.4,  
  
    amp=True,  
)
```

ko‘rinishi tushunarliroq bo‘ladi.

Bunda:

dastlabki 4 epoch – Mosaic ishlaydi
oxirgi 1 epoch – Mosaic o‘chadi

maqsadi ko‘zda tutiladi.

Augmentation ishlayotganini qanday ko‘rish mumkin?

Training natijalari papkasida ko‘pincha:

train_batch0.jpg

train_batch1.jpg

train_batch2.jpg

kabi rasmlar yaratiladi.

Ularini ochganda:

- Mosaic birlashtirilgan rasmlar;
- rang o'zgarishlari;
- flip;
- masshtab o'zgarishi;
- bounding boxlar

ko'rinadi.

Bu rasmlar augmentation va annotationlar bir-biriga mosligini tekshirish uchun juda muhim.

Umumiy xulosa

Tushuncha	Asosiy vazifa
Data Augmentation	Mavjud rasmlarning turli ko'rinishlarini yaratish
Mosaic	To'rtta rasmni birlashtirib, kichik obyekt va turli masshtablarni o'rgatish
Manual Annotation	Inson tomonidan aniq box va class belgilash
Auto-Label	AI yordamida tez dastlabki annotation yaratish
Dataset Version	Datasetning o'zgarmaydigan, qayta ishlatiladigan holatini saqlash
ZIP	Datasetni qo'lda yuklash
API	Datasetni kod orqali avtomatik va aniq version bilan yuklash
Training augmentation	mosaic, fliplr, scale, hsv_* kabi parametrlar
augment=True	Joriy rasmiy sozlamalarda predict/validation uchun TTA
amp=True	GPU'da mixed-precision trainingni yoqish